Dida-Trade

Project Final Report – Group 43

Rafael de Oliveira Gomes (No. 110285)
Francisco Teles do Carvalhal dos Santos Andrade (No. 110720)
Tomás Branco de Paiva de Jesus Rodrigues (No. 120492)
Instituto Superior Técnico
MEIC-A
Design and Implementation of Distributed Applications
Academic Year 2026–2027

October 16, 2026

Abstract

We implemented dida-trade, a trade application replicated with Paxos, covering all five steps of the project. This report describes the parts of our design that are not obvious from the specification: how the leader runs Phase 1 once per ballot while pipelining instances (Multi-Paxos), how servers propose requests directly in fast ballots and recover from collisions (Fast Paxos), and how `balance` *is served without consensus while preserving causal order, using the log position of decided requests as a scalar causal clock.*

1 Implemented Steps

All the steps of the project were implemented, so we target the *Ultimate* level for grading. Steps 1 and 2 were already delivered at the checkpoint.

Step	Done	Delivered
0 Baseline	provided	–
1 Phase 1 and debug	yes	checkpoint
2 Multi-Paxos	yes	checkpoint
3 Fast Paxos	yes	final delivery
4 Causal balance	yes	final delivery
5 Dynamic membership	yes	final delivery

2 Code Overview

2.1 Modules

- *core*: `TradeManager` keeps the users and the FIFO queues of buy and sell orders, and matches them.
- *configs*: one schedule per configuration (A, B, C) gives, for each ballot, its acceptors, learners, leader and quorum, and whether it is fast.
- *contract*: three gRPC services, `DidaTradeMain` (client–server), `DidaTradePaxos` (server–server: Phase 1, Phase 2 and learn) and `DidaTradeMaster` (console–server).
- *util*: collectors that gather asynchronous replies and decide when enough have arrived.
- *server*, *app* (client) and *console*: the executables.

2.2 Server structure

Each server runs three gRPC services: one for clients, one acting as acceptor and learner (`DidaTradePaxosServiceImpl`) and one for the console. They share a `DidaTradeServerState`, which holds the ballots, the instance cursors, the trade manager, the Paxos log (one entry per instance, with the accepted value and ballot and a *decided* flag) and the `RequestHistory` (one record per client request, pending or processed). Two threads run on this state, the Proposer and the Executor (Step 2).

2.3 Life of a request

A `buy` in a classic ballot goes through the code as follows:

1. `DidaTradeApp.buy` sends the request to every server and waits for the first reply.
2. On each server, `DidaTradeMainServiceImpl.buy` records the request in the `RequestHistory`, wakes the Proposer and blocks until the request has been executed.
3. On the leader, `Proposer.proposeInstance` reserves an instance, runs Phase 1 if needed, and sends Phase 2 with the request id as the value.

Step	Class	Methods and fields
1 Phase 1, debug	`PhaseOneResponseProcessor`	`onNext` (quorum, early stop)
	`DidaTradeMasterServiceImpl`	`setdebug`
	`DidaTradeServerState`	`checkDebugState`
2 Multi-Paxos	`Proposer`	`run`, `hasWorkToDo`, `proposeInstance`, `sendPhaseOne`, `fillHoles`, `sendPhaseTwo`
	`Executor`	`run`, `executeEntry`
	`DidaTradeServerState`	instance cursors, `setCurrentBallot` (resets Phase 1 state)
	`DidaTradePaxosServiceImpl`	`phaseone` (returns adopted entries)
	`RequestHistory`	`getFirstNotProposed`, `waitForPending`, `resetProposedFlags`
3 Fast Paxos	`DidaTradeMainServiceImpl`	`decideRequestPath`, `sendFastDirect`
	`DidaTradePaxosServiceImpl`	`phasetwo` (acceptor rule, `setFastPaxosMode`), `learn` (`accepts_by_value`)
	`Proposer`, `PaxosInstance`	`sendPhaseOne` (proposes `ANY_VALUE`)
4 Causal balance	`DidaTradeApp`	`updateSlot`, `balance` (`last_seen_slot`)
	`DidaTradeMainServiceImpl`	`balance`
	`DidaTradeServerState`, `Executor`	`waitForMinSlot`, `getAppliedSlot`; `executeEntry` (`setSlot`)
	`DidaTradeMain.proto`	`slot`, `min_slot`, `applied_slot`
5 Dynamic membership	`Proposer`	`sendPhaseOne` (groups), `transferState`, `sendPhaseTwoTracked`
	`PhaseOneResponseProcessor`	constructor with groups and quorums
	`DidaTradePaxosServiceImpl`, `ScheduleC`	`phasetwo` (`isacceptor`); `quorum`; `NOOP_VALUE`

Table 1: Code map: the classes and methods each step adds or changes.

4. Each acceptor accepts it in `phasetwo` and sends a learn message to all learners.

5. When `learn` counts a quorum for one value, the instance is decided and the Executor is woken.

6. `Executor.executeEntry` applies the command, in instance order, and unblocks the handler of step 2, which replies.

Table 1 lists, for each step, the classes and methods it adds or changes. In a fast ballot, step 3 is done by every server (Step 3); a `balance` skips steps 2–6 (Step 4).

3 Step 1: Phase 1 and Debug Modes

3.1 Goal

Replace the deliberately wrong Phase 1 of the baseline with a correct one, and add debug modes to inject failures from the console.

3.2 The bogus Phase 1

`PhaseOneBogusProcessor` stopped after the *first* reply, always reported success and took the value from that single reply. The leader thus ran Phase 2 without a quorum of promises and could miss a chosen value. *Scenario:* A is decided in instance 0 by acceptors 0 and 1. A later leader's first Phase 1 reply comes from acceptor 2, which never saw A; the leader proposes B with a higher ballot, every acceptor accepts it, and replicas diverge.

3.3 Correct Phase 1

The leader sends Phase 1 to every acceptor and waits while `PhaseOneResponseProcessor.onNext` handles each reply: it counts promises and keeps the accepted value with the highest ballot, which the leader then proposes (or its own request, if there is none). The leader is released as soon as the outcome is known: a quorum of promises (*success*); a quorum no longer reachable (*failure*); or one acceptor that promised a higher ballot (*rejection*: the higher ballot would replace this round anyway, so the leader adopts it at once). Stopping early only affects liveness: Phase 2 still requires a quorum of promises, and any two quorums intersect.

3.4 Debug modes

The console's command reaches `DidaTradeMasterServiceImpl.setdebug`, which stores the mode in the server state. *Crash* exits the process. Every client and Paxos handler starts with `checkDebugState`: while *frozen* it blocks until *unfreeze* wakes it; in *slow mode* it sleeps a random 0–2 s. The console service never calls it, so a frozen server can always be unfrozen.

4 Step 2: Multi-Paxos

4.1 Goal

The baseline `MainLoop` ran one instance at a time, from Phase 1 to execution. Step 2 runs Phase 1 once per ballot and keeps several instances in flight.

4.2 Proposer and Executor

Proposer. `Proposer.run` sleeps until this server leads the current ballot and has work to do (Phase 1 not done yet, or a request not yet proposed). It then reserves the next instance and calls `proposeInstance`. Phase 2 is sent with asynchronous stubs and the replies are handled in a callback, so the Proposer moves on to the next instance without waiting (pipelining).

Executor. `Executor.executeEntry` waits until instance `next_instance_to_execute` is decided (the learn handler wakes it), applies the command, answers the client and advances the cursor. Decisions may arrive out of order; execution never does. The Paxos value is only the request id: since clients broadcast, every replica gets the request itself. If the decision arrives first, the Executor blocks in `waitForPending` until the request is registered.

4.3 Phase 1 once per ballot

The flag `phase1_done_for_ballot` says whether the leader has completed Phase 1 in its current ballot. If not, `proposeInstance` first calls `sendPhaseOne`, starting at the instance it just reserved. Phase 1 covers *all* instances from there: in `phaseone`, an acceptor returns every value it has accepted from that instance on, as (instance, value, ballot) entries in the new `adopted` field of the reply. For each instance the leader keeps the value with the highest ballot in `adopted_by_instance`, and sets the flag; later instances go straight to Phase 2. Our first version covered a single instance; with pipelining, a value already chosen in a *later* instance could then be overwritten by a new leader. Every ballot increase goes through `setCurrentBallot`, which clears the flag and the map, so the next instance runs Phase 1 again.

4.4 Choosing the value of an instance

`proposeInstance` takes the first case that applies: (1) instance already decided: skip it; (2) a value was adopted for it: re-propose that value; (3) a pending request not yet proposed: propose it and set its `proposed` flag; (4) nothing to propose: release the instance. Right after Phase 1, `fillHoles` proposes a no-op (`NOOP_VALUE`) in every instance up to the highest adopted one that nobody had accepted, so the Executors never wait at an empty slot.

4.5 Failures and retries

If Phase 1 is rejected, the leader adopts the higher ballot, releases the instance and backs off 200 ms. If a Phase 2 can no longer reach a quorum, the callback clears the request's `proposed` flag so it is proposed again. On a new ballot, the console's handler calls `resetProposedFlags`, but *only* on the new leader: doing it on every Phase 2 rejection would release unrelated in-flight requests, which could then be decided twice.

4.6 Auxiliary structures

- `next_instance_to_propose` (leader): next free instance; when an instance is released it is decremented, but only if no later one was taken.
- `next_instance_to_execute` (all): next instance to apply.
- `phase1_done_for_ballot`, `adopted_by_instance` (leader): set by Phase 1; cleared on every ballot increase.
- `proposed` flag (all): avoids proposing a request twice.
- `RequestHistory` (all): ordered map, so pending requests are proposed in arrival order.

5 Step 3: Fast Paxos

5.1 Goal

In a fast ballot, servers propose client requests directly to the acceptors instead of waiting for the leader, so a request is decided in one round trip. Two servers may then propose different requests for the same instance (a *collision*), which must be resolved.

5.2 Opening a fast ballot

The leader runs Phase 1 as usual. Then, if the schedule marks the ballot as fast and no value was adopted for the instance, it proposes the special value `ANY_VALUE`. An acceptor that accepts it sets `fastpaxos_on`, so its server only uses the fast path once the acceptors are ready. The `ANY` instance is decided normally and skipped by the Executor.

5.3 Fast path

Every client handler calls `decideRequestPath`. If fast mode is off, the request goes to the Proposer as in Step 2. Otherwise `sendFastDirect` takes the next instance from the server's own `next_instance_to_propose` and sends Phase 2 directly, with the current ballot. Since every server receives every request, when they see requests in the same order they propose the same value for the same instance, and it is decided without the leader (see the Evaluation for how this behaves in practice).

5.4 Acceptor rule

In `phasetwo`, an acceptor accepts a proposal only if its ballot is higher than the one already accepted for that instance, or equal *with the same value* (a repeat from another server). A different value at the same ballot, or for an instance already decided, is rejected. The check and the update run under a lock, so two concurrent proposals cannot both pass.

5.5 Deciding a value

Different acceptors may accept different values for one instance, so `learn` counts votes *per value* in `accepts_by_value`. The instance is decided only when one value reaches a quorum. A learn message with a higher ballot discards the votes of older ballots.

5.6 Collision recovery

The losing server notices the collision in its Phase 2 callback, once the rejections make a quorum impossible. It releases the instance, clears the request's `proposed` flag and wakes its Proposer: the request falls back to the classic path and the leader proposes it in a later instance. The ballot does not change.

5.7 Quorum and closing fast mode

Fast rounds use the classic quorum (2 of 3 in schedules A and B, 3 of 5 in C). Every ballot increase turns fast mode off, so each ballot starts classic until its leader proposes `ANY` again.

5.8 Auxiliary structures

- `fastpaxos_on` (each server): set when its acceptor accepts `ANY`; cleared on every ballot increase.
- `ANY_VALUE` (constant): opens fast mode; skipped by the Executor.
- `accepts_by_value` (learner, per instance): votes per value in the current ballot.

6 Step 4: Causal Balance

6.1 Goal

Serve `balance`, which does not change the state, from any replica without consensus, while still reflecting everything the client has observed (read-your-writes and monotonic reads).

6.2 Slot as a causal clock

All writes are totally ordered in the log and executed in order, so a replica's state is a function of the prefix it has executed. A single integer, the *slot* (instance), is enough as causal context.

Client. Every write reply now carries the slot where the request was executed (also if it was rejected), and every balance reply the replica's `applied_slot`. The client keeps the highest slot seen in `last_seen_slot`, and `DidaTradeApp.balance` sends it as `min_slot` to all servers; the first reply is used.

Server. The applied slot is `next_instance_to_execute` minus one, which only advances in order. `DidaTradeMainServiceImpl.balance` calls `waitForMinSlot`, which blocks until `min_slot` has been applied (the Executor wakes it at every step), then reads the balance directly from the trade manager, bypassing the Proposer, and returns the applied slot.

Executor order. `executeEntry` records the slot in the request, advances the cursor, and only then answers the client: a replica that replies with slot n has already counted n as applied.

6.3 Slots that carry no request

`ANY` markers and no-ops are decided but carry no request. The Executor applies nothing for them but still advances the cursor; otherwise it would wait on that instance forever and no later balance could be answered. Clients may therefore see gaps in slots, which is harmless: a marker leaves the state unchanged, so "executed up to n" means the same state on every replica.

6.4 Guarantees and limits

Each client gets read-your-writes and monotonic reads, and any replica may answer, so `balance` works with the leader frozen or crashed; a replica that is behind just answers later. Writes of other clients with no causal link may not be visible, as the specification allows.

6.5 Auxiliary structures

- `last_seen_slot` (client): highest slot seen in any reply; only increases.
- slot of each request (all): set by the Executor, returned in the reply.

7 Step 5: Dynamic Membership

7.1 Goal

In Schedule C (seven servers) all servers are always learners, but the acceptors are $\{S_0..S_4\}$ in configurations 0–1 and $\{S_2..S_6\}$ in 2–4. A value chosen by a majority of the old acceptors may be unknown to a majority of the new ones, so on a change we must (i) read what the old acceptors accepted and (ii) copy it into the new ones.

7.2 Roles and quorums per configuration

Each configuration uses its own majority (3 of 5). `phasetwo` rejects a request when the server is not an acceptor of that ballot; when the ballot is below the server's promise (which *seals* the old configuration against a deposed leader); or when the instance is decided with a *different* value (the same value is accepted, which makes the copy possible). Learn messages still go to all seven servers, so former acceptors keep executing the log and serving clients.

7.3 Phase 1 over several configurations

`sendPhaseOne` now builds one group of acceptors per distinct configuration between the last *completed ballot* and the current one, each with its own quorum. It sends Phase 1 to their union and succeeds only when *every* group reaches its quorum (a majority of the union might miss a majority of the old set). With one group it asks from its first free instance, as in Step 2; with several it asks from instance 0, since the new acceptors need the whole log.

7.4 State transfer

`transferState` sends, for every instance up to the highest one found, a Phase 2 with the leader's ballot carrying the value found, or `NOOP_VALUE` for a hole. All are sent at once, and the leader waits (up to 10 s) for a majority of the new acceptors on each one; otherwise Phase 1 runs again. Phase 2 is used rather than learn because a future leader reads what acceptors *accepted*, and only Phase 2 sets that. Example: in configuration 2, S_3 reads majorities of $\{S_0..S_4\}$ and $\{S_2..S_6\}$ from instance 0 and copies the log into $\{S_2..S_6\}$; afterwards S_0 and S_1 may crash.

7.5 Finishing the change

After the copy the leader (1) sets `next_instance_to_propose` to the last copied instance plus one, because fast-path proposals may have moved it further and left holes that would block the Executor; (2) marks as proposed every pending request already in the copied log, so it is not decided twice; (3) marks the ballot as completed, which lets the console reply and bounds later Phase 1s, and in a fast configuration proposes `ANY` at the next instance.

7.6 Auxiliary structures

- *Completed ballot* (servers, console): last ballot whose configuration was fully carried over; the console sends it with each new ballot, and it is the lower bound of the Phase 1 groups.
- *Adopted map* (leader): content of the transfer.
- `NOOP_VALUE` (log): fills holes; skipped by the Executor.

7.7 Limits

The transfer copies the whole log each time the acceptor set changes; a snapshot plus the log tail would avoid that. Fast configurations decide with 3 of 5, not the fast quorum $\lceil 3n/4 \rceil = 4$. Stress runs with concurrent clients and a slow server kept all replicas identical, but this is testing, not a proof.

8 Evaluation

We tested by hand, running all processes on one machine and saving the output of each one to a file. After each scenario we compared, on every replica, the sequence of executed requests and the state printed by `show`; since the state machine is deterministic, identical sequences must give identical states.

- *Concurrent clients* (fast and classic ballots, one server in slow mode): all servers executed the same instances in the same order and ended with the same state, even though several competing proposals were rejected.
- *Frozen or crashed servers*: with the leader frozen or crashed, writes wait but `balance` is still answered; after a ballot change the pending writes complete once, and a returning old leader is rejected by the promise check. Crashing replicas that are not the leader does not stop the others from deciding and executing requests, as long as a majority of the acceptors is alive.
- *Schedule C*: after the change to configuration 2, S_5 and S_6 hold the whole old log, and S_0 and S_1 keep executing and answering clients. After crashing both and starting configuration 3, Phase 1 reads only $\{S_2, \ldots, S_6\}$ and all balances are preserved.
- *Causal balance*: balances never enter the log and always reflect the client's last write, even when a slow replica answers.

Manual testing does not prove correctness. Two limitations remain: in fast ballots the direct proposals of servers other than the leader are often rejected and fall back to the classic path, and a replica that misses the messages of an instance cannot catch up, so server 0 is started last.

9 Conclusion

We extended the baseline into a replicated trade service that uses Multi-Paxos with pipelining, Fast Paxos, causally ordered `balance` requests answered without consensus, and dynamic membership. The main lesson was that each optimization is simple on its own; the bugs we found came from how they interact with leader changes. Pipelining required Phase 1 to cover every instance from the leader's current one onwards, and resetting the `proposed` flags whenever a ballot changed was enough to let the same request be decided twice. Step 4 showed the opposite: because the log already orders all writes, a single integer was enough to give each client read-your-writes and monotonic reads.

10 Use of AI

All design choices and the code have been made by us. The following tools have been used for this purpose: *Claude Code* and *Claude Chat* (by Anthropic), for interpreting the project specification, helping debug code, suggesting implementations, interpreting results, and helping to write this report.

11 Authorship

We,

Rafael Gomes, n.º 110285, MEIC-A
Francisco Andrade, n.º 110720, MEIC-A
Tomás Rodrigues, n.º 120492, MEIC-A

We hereby declare, on our honor, that the work entitled "Dida-Trade", carried out as part of the Design and Implementation of Distributed Applications course, resulted from the active participation of the following group members. The individual contributions were:

- Rafael Gomes – Step 4: Causal Balance; Step 5: Dynamic Membership
- Francisco Andrade – Step 1: Phase 1; Step 2: Multi-Paxos
- Tomás Rodrigues – Step 1: Phase 1; Step 3: Fast Paxos; Step 5 Dynamic Membership

We declare that this work is original and complies with the applicable academic regulations.
Lisbon, October 2026
Signatures: